# React + TypeScript con Vite

## Base profesional para desarrollo moderno de interfaces

### 1. Introducción al Stack Moderno

En el desarrollo frontend actual, la combinación **Vite + React + TypeScript** representa una de las bases más sólidas y profesionales para construir aplicaciones escalables.

Este stack resuelve tres problemas históricos:

- Lentitud en herramientas tradicionales
- Falta de tipado en JavaScript
- Desorganización estructural en proyectos React

El objetivo no es solo “hacer que funcione”, sino construir desde el inicio una arquitectura limpia y mantenible.

### 2. Herramientas del Stack

#### 2.1 Vite

Vite es un bundler y servidor de desarrollo moderno.

Características principales:

- Arranque instantáneo
- Hot Module Replacement (HMR)
- Configuración mínima
- Optimización automática para producción

Reemplaza herramientas más antiguas como Create React App, que presentan tiempos de compilación elevados.

### 2.2 React

React es una biblioteca para construir interfaces de usuario basadas en componentes reutilizables.

Principios fundamentales:

- UI declarativa
- Componentes independientes
- Composición
- Reutilización

React no es un framework completo; se centra exclusivamente en la capa de vista.

### 2.3 TypeScript

TypeScript es un superset de JavaScript que añade tipado estático.

Ventajas:

- Detección de errores en tiempo de compilación
- Autocompletado avanzado
- Documentación implícita
- Contratos claros entre componentes

Trabajar sin tipos no es un problema del programador; es un problema arquitectónico.

### 2.4 pnpm

pnpm es un gestor de paquetes eficiente para Node.js.

Ventajas:

- No duplica dependencias en disco
- Más rápido que npm en proyectos medianos y grandes
- Manejo optimizado de node_modules

En esta base se utiliza exclusivamente pnpm.

# 3. Instalación del Proyecto

## 3.1 Crear el proyecto

Desde la terminal:

```
pnpm create vite
```

Seleccionar:

- Framework: React
- Variant: TypeScript

## 3.2 Instalar dependencias

```
cd nombre-proyecto
```

```
pnpm install
```

Esto instala todas las dependencias definidas en package.json.

## 3.3 Ejecutar el servidor de desarrollo

```
pnpm dev
```

El proyecto quedará disponible en:

http://localhost:5173

## 4. Estructura del Proyecto

Al crear el proyecto, se genera la siguiente estructura:

```
nombre-proyecto/
│
├── index.html
├── package.json
├── tsconfig.json
├── vite.config.ts
│
└── src/
   ├── main.tsx
   ├── App.tsx
   └── assets/
```

### 4.1 Archivos en la raíz

**index.html**

Contiene el contenedor principal:

```
<div id="root"></div>
```

React renderiza la aplicación dentro de ese div.

**vite.config.ts**

Archivo de configuración del bundler.
No se modifica en etapas iniciales.

**tsconfig.json**

Configuración del compilador TypeScript.
Define reglas como:

"strict": true

Activa el tipado estricto.

## 5. Carpeta src

Todo el código fuente vive dentro de src/.

Regla estructural:

Nunca desarrollar lógica fuera de src.

### 5.1 main.tsx

Es el punto de entrada de la aplicación.

Ejemplo:

```
import React from "react"
import ReactDOM from "react-dom/client"
import { App } from "./App"

ReactDOM.createRoot(
 document.getElementById("root")!
).render(
 <React.StrictMode>
   <App />
 </React.StrictMode>
)
```

Responsabilidades:

- Crear el root de React
- Renderizar el componente principal
- Activar StrictMode

No se modifica habitualmente.

## 5.2 App.tsx

Es el componente raíz de la aplicación.

Ejemplo básico:

```
export const App = () => {
 return (
   <div>
     <h1>Mi aplicación</h1>
   </div>
 )
}
```

Desde aquí se compone el resto de la UI.

## 6. JSX

JSX es una extensión sintáctica que permite escribir estructuras similares a HTML dentro de JavaScript.

Ejemplo:

```
const titulo = <h1>Hola mundo</h1>
```

Internamente se transforma en:

```
React.createElement("h1", null, "Hola mundo")
```

### 6.1 Reglas de JSX

- Un único elemento raíz
- className en lugar de class
- Uso de {} para expresiones JavaScript

Ejemplo:

```
const nombre = "Luciano"

<h1>Hola {nombre}</h1>
```

## 7. Componentes Funcionales

Un componente es una función que retorna JSX.

En esta base se utilizan exclusivamente:

- Arrow functions
- Constantes
- Exportaciones nombradas

Ejemplo:

```
export const Saludo = () => {

 return <h2>Bienvenido</h2>

}
```

### 7.1 Convenciones

- PascalCase para nombres
- Un componente por archivo
- Nombre del archivo igual al componente
- Export nombrado (export const)
- Extensión .tsx

## 8. Organización por Carpetas

Estructura recomendada:

```
src/
│
├── App.tsx
├── main.tsx
│
└── components/
   ├── Saludo.tsx
   └── Card.tsx
```

Todos los componentes reutilizables viven en components/.

## 9. Props

Las props permiten que un componente reciba datos desde el exterior.

Son el mecanismo de comunicación entre componentes.

## 10. Props Tipadas con TypeScript

El tipado se realiza utilizando type.

**Paso 1: Definir el contrato**

```
type SaludoProps = {
 nombre: string
}
```

**Paso 2: Aplicar el tipo**

```
export const Saludo = ({ nombre }: SaludoProps) => {
 return <h2>Hola {nombre}</h2>
}
```

**Paso 3: Usar el componente**

```
<Saludo nombre="María" />
```

Si se omite nombre, TypeScript genera error de compilación.

## 11. Componente Reutilizable – Card

Ejemplo práctico:

```
type CardProps = {
 titulo: string
 descripcion: string
}

export const Card = ({ titulo, descripcion }: CardProps) => {
 return (
   <div>
     <h3>{titulo}</h3>
     <p>{descripcion}</p>
   </div>
 )
}
```

Uso:

```
<Card
 titulo="React"
 descripcion="Biblioteca para construir interfaces"
/>
```

## 12. Composición

React se basa en composición.

Un componente puede contener otros componentes:

```
export const App = () => {
 return (
   <div>
     <Saludo nombre="Luciano" />
     <Card titulo="React" descripcion="UI declarativa" />
   </div>
 )
}
```

Esto permite:

- Reutilización
- Modularidad
- Mantenimiento centralizado

## 13. Buenas Prácticas Iniciales

1. No usar any
2. Siempre tipar props
3. Usar export nombrado
4. Respetar PascalCase
5. Mantener estructura organizada
6. No mezclar lógica fuera de src

## 14. Errores Comunes

❌ No tipar props
❌ Usar any
❌ Export default anónimo
❌ No respetar mayúsculas
❌ Escribir class en JSX

## 15. Conclusión

La combinación Vite + React + TypeScript no es una moda; es una mejora estructural.

Permite:

- Escalabilidad
- Seguridad de tipos
- Organización clara
- Base profesional desde el inicio

Una arquitectura sólida reduce errores futuros.

Primer orden.
Después la complejidad.